
UNIVERSIDAD DE CANTABRIA

INGENIERÍA DEL SOFTWARE II

PRÁCTICA 6

Alejandro López-Tormos Gervás

Date created:
Sábado, 6 de mayo de 2026

REFACTORIZACIONES

1. Código antiguo

Métricas de Refactorización

Nombre clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
gestionTransportes	6	2	4	1.3	2	0	0
Conductor	18	2	9	1	5	0	0
Transporte	9	1.8	4	0.8	4	0	0
CategoriaTransporte	0	0	0	0	3	0	0
GestiónTransporteGUI	18	9	34	17	4	0	0

Clases que contribuyen al CBO:

- gestionTransportes: GestionTransportesGUI, Conductor
- Conductor: IllegalArgumentException(), gestiónTransportes, GestionTransportesGUI, Transporte, CategoriaTransporte
- Transporte: CategoriaTransporte, IllegalArgumentException(), GestionTransportesGUI, Conductor
- CategoriaTransporte: Conductor, Transporte, GestionTransportesGUI
- GestiónTransporteGUI: CategoriaTransporte, Transporte, Conductor, gestionTransportes

Cambios clase Transporte

El nombre de la variable ton no es explicativo porque he tenido que comprender el código para deducir que se refiere a las toneladas que lleva el camión. También se cambia el getter ton() a getToneladas().

Hay getters que no llevan get delante; se refactorizan añadiendo get al principio del nombre del método.

La variable valor es confusa porque puede referirse tanto a personas como a toneladas. Se va a resolver introduciendo herencia en Transporte para eliminar esa ambigüedad. Como consecuencia, se elimina el if que evalúa la categoría del transporte.

Al haber modificado métodos de clases, se corrigen los fallos que aparezcan en el resto de clases por estos cambios.

Cambios clase Conductor

dire es un nombre informal, por eso se cambia a direccion, e igual pasa con su getter.

Hay dos getters del DNI que hacen lo mismo pero con diferente nombre; se elimina el que no sigue el estándar de Java.

Falta el get en el getter de apellido2.

Se puede extraer el switch a una función auxiliar, ya que se encarga de calcular el sueldo extra. Aún mejor, se puede eliminar el switch definiendo un método en las clases pertinentes. Como la única diferencia entre MercanciasPeligrosas y Mercancias es un único valor en esta función, no se considera conveniente añadir más herencia y se resuelve con un if.

Como el método extraído es demasiado simple, se puede eliminar y volver a integrarlo en una línea dentro de la función original para calcular el sueldoExtra.

Por último, se eliminan los números mágicos y se expresan como constantes.

Cambios clase gestionTransportes

Se cambia el nombre de `gestionTransportes` por `GestionTransportes` para cumplir el estándar de Java.

La variable `cs` tampoco es muy explicativa, se ha decidido cambiarla por `conductores`.

Se puede optimizar el código si, en vez de una lista, los conductores se almacenan en un `HashMap` con el DNI como clave y el conductor como valor.

Cambios clase `CategoriaTransporte`

Para cumplir la convención de Java, los valores del enumerado se ponen en mayúsculas.

Cambios clase `GestionTransportesGUI`

Se cambia el nombre de las variables `lect`, `c`, `gt` y `t` a `lectura`, `conductor`, `gestionTransportes` y `transporte` para que sean más explicativas. Para renombrar `c`, hay que cambiar también el nombre de la variable `conductor` dentro de los `for-each`; por convención en los `for-each` se usa una sola letra, en este caso `c`.

En vez de definir las opciones del menú como variables locales dentro de `main`, se sacan fuera como constantes de clase. En este caso no merece la pena crear un enumerado.

Como el `main` es demasiado grande, cada caso del `switch` se extrae a una función auxiliar.

Se hace que las clases del modelo devuelvan un `String`, así no hay que repetir la llamada a `mensaje` en cada función, ya que solo se llama una vez por función.

Hay mensajes de error, pero no hay mensajes de que todo haya salido correctamente, lo que puede resultar confuso para el cliente al no saber si la operación se completó.

El método `mejorConductor` debe estar en la clase `GestionTransportes`, ya que solo utiliza la lista de conductores y métodos propios. Además, se optimiza el código porque se llama varias veces al mismo método (`suelto` ()) cuando podría guardarse el resultado en una variable. Por otro lado, al ser un método muy largo, se ha decidido separar la parte que formatea el `String` con el resultado final en una función privada. Adicionalmente, parece haber un bug en el formateo del mensaje, ya que se imprime dos veces el nombre del conductor separados por un espacio en lugar de mostrar nombre y apellido.

En el método `sueltoConductor` se separa la parte de lectura de datos de la parte de encontrar al conductor y calcular su sueldo. Esta segunda parte se mueve a la clase `GestionTransportes`.

Además, se invierte el orden del `if` para usar un `early return`.

En el método `anhadeTransporte` la variable `t` sobra; pueden realizarse todas las operaciones necesarias sin ella.

Hacer el `switch` con cadenas para identificar la categoría del transporte no es lo más correcto; es mucho mejor hacer el `switch` con el enumerado. Además, si el tipo es nulo se implementa un `early return`.

Se incluye un `case` en el `switch` para poder salir del bucle `while` y un `default` que no hace nada.

Se eliminan los imports de `LinkedList` y `List`, ya que ya no se usan en esta clase.

Tests

Se elimina `TransporteTest` porque ahora `Transporte` es una clase abstracta; los tests correspondientes se trasladan a las clases hijas.

Se quitan las comprobaciones cruzadas (verificar que un `TransporteMercancias` tiene 0 personas, o que un `TransportePersonas` tiene 0 toneladas), ya que con la nueva jerarquía esos atributos ya no existen en la clase contraria.

Se crean TransporteMercanciasTest y TransportePersonasTest con los tests propios de cada subclase: validación del constructor, getters y cálculo del sueldoExtra para los distintos casos.

Se crean los test para cubrir el 100% de la cobertura de la clase Conductor.

2. Código Refactorizado

Métricas de Refactorización

Nombre clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
GestionTransportes	19	2.71	16	2.29	5	0	0
Conductor	13	1.6	3	0.4	4	0	0
Transporte	5	1.25	1	0.25	3	0	2
TransporteMercancias	6	1.5	2	0.5	3	1	0
TransportePersonas	6	1.5	3	0.75	3	1	0
CategoríaTransporte	5	5	2	2	5	0	0
GestiónTransporteGUI	13	2.6	5	1	2	0	0

Clases que contribuyen al CBO:

- GestionTransportes: Conductor, CategoríaTransporte, GestionTransportesGUI, TransporteMercancias, TransportePersonas.
- Conductor: IllegalArgumentException(), Transporte, CategoríaTransporte, GestionTransportes
- Transporte: CategoríaTransporte, IllegalArgumentException(), Conductor
- TransporteMercancias: IllegalArgumentException(), CategoríaTransporte, GestionTransportes
- TransportePersonas: IllegalArgumentException(), CategoríaTransporte, GestionTransportes
- CategoríaTransporte: GestionTransportes, GestionTransportesGUI, Transporte, TransporteMercancias, TransportePersonas
- GestiónTransporteGUI: CategoríaTransporte, gestionTransportes

Como se puede ver las métricas normalizadas han disminuido en gran medida en la mayoría de las clases menos en el enumerado al tener un método más con un switch.

Esto quiere decir que las métricas de refactorización han mejorado enormemente la calidad del código, haciendo que ahora sea más fácil de leer y entender. Aparte, aunque no influya en las métricas de refactorización se han cambiado nombres de variables y la estructura del código para seguir el estándar de Java. Por último, también se han implementado mejoras en el código que quizá aumenten estas métricas pero que merece la pena implementar.

Diagrama de clases final

